ПРОТОКОЛ ОБМЕНА: БОРТОВОЙ ПК ↔ ПЛАТЫ РОБОТА

Единственный протокол плат робота — **текстовый, построчный, по USB-CDC/UART**. Никаких бинарных заголовков, кадров фиксированной длины и CAN/RS-485 в проекте нет: каждая плата Arduino Mega 2560 / Mega Mini подключена к бортовому компьютеру **своим USB-кабелем** и обменивается строками, заканчивающимися переводом строки.

Плат пять, прошивок две:

Плата	Скетч (источник истины)	Версия	Разделы
4 × контроллер колеса (FL, FR, RL, RR)	`firmware/motor_wheel/motor_wheel.ino`	**fw26**	§1–§6
модуль датчиков: свет, замок отсека, карты, компас	`firmware/sensor_module/sensor_module.ino`	протокол **4**	§7

Документ описывает то, что реально делают прошивки; описание команд монитора, калибровок и схемы включения — в `firmware/README.md`. Остальные устройства на USB бортового компьютера (лидар, BMS, принтер, камеры) этим протоколом не пользуются — они перечислены в §8.

Параметры обмена с платами колёс:

Параметр	Значение
Физика	USB (CDC) или UART, **115200 8N1**
Кодировка	ASCII, кириллица только в человекочитаемых строках `#  …` (UTF-8)
Разделитель команд	`\n` (монитор порта: «NL» или «NL & CR»)
Одна плата = одно колесо	`MODULE_ID` = FL / FR / RL / RR
Роль в телеметрии	поле `mod=` (`@TLM`) и 2-е поле `@F` — цифровой код 1…4
Контроль целостности	**CRC16/Modbus** (полином `0xA001`, init `0xFFFF`), hex-суффикс `crc=` / 4 hex-цифры
Сторож привода	нет команды тяги **650 мс** → выход 0 и тормоз
Короткая телеметрия	`@F` каждые ~30 мс (не чаще 20 мс)
Полная телеметрия	`@TLM` каждые 200 мс
Ответ на запись нуля	`@Z` (только после `zq`/`zc`)

---

1. Направление ПК → контроллер: строки команд

Команда — обычная текстовая строка, например `w FL 12.5`. Сервер всегда шлёт **адресную** форму: имя колеса присутствует, а сама плата получает уже короткую строку без имени (имя удаляется сервером до отправки). В мониторе порта видны обе формы: прошивка принимает своё имя и **игнорирует чужое**.

1.1. Руль

Команда	Назначение
`w [XX] <угол>`	абсолютный угол колеса, °; нужен заданный ноль; предел **±100°**
`w [XX] zero`	ручной ноль в текущем положении
`w [XX] calib`	калибровка по оптодатчику D18
`w [XX] z` / `w [XX] k`	короткие формы `zero` / `calib`
`k` / `calib`	калибровка **всех** колёс (рассылается на все порты)
`z` / `zero`	ручной ноль без имени
`h`	в ноль
`d <угол> [f/b]`	относительный поворот, цель в пределах ±125°
`c <угол>` / `c c`	качание / выключить качание
`g`	продолжить поворот, прерванный `s`
`s`	СТОП всего: руль, калибровка, качание, тяга

1.2. Тяга

Команда	Назначение
`v [XX] <0…150> [f/b]`	прямое значение `Servo.write`, без пересчёта; `f` по умолчанию
`v [XX] 0`	останов и тормоз
`t [XX] <−100…+100>`	прежний процентный режим (1 % ≈ 1,8 значения шкалы)
`b`	тормоз всех BLDC
`b <id>`	тормоз с идентификатором для подтверждения (`brake_id`)
`n <id>`	**начиная с fw25:** не отпускает тормоз, а вызывает тормоз (в fw18…24 — нейтраль)

1.3. Датчик положения и служебные

Команда	Назначение
`q`	калибровка потенциометра A0: 0°, −60°, +60° (включает доводку)
`pm 1` / `pm 0`	режим «с потенциометром» / «без потенциометра» (сбрасывает `q`)
`ph 1` / `ph 0`	активное удержание угла вкл/выкл (fw24+); при потере сигнала A0 плата fw26 выключает его сама
`j <id> <seq> <тяга> <угол>`	служебное руление на ходу (fw20+)
`s q`	тот же STOP, но без обычной заметки о паузе (fw21+)
`zq <id>`	проверка сохранённой поправки полного нуля (fw23+)
`zc <id>`	запись поправки полного нуля в EEPROM (fw23+)
`zr CONFIRM`	сброс неисправной сохранённой поправки
`p`	состояние колеса (человекочитаемый отчёт)
`?` / `help`	справка

1.4. Что проверяет плата до выполнения

Прошивка отклоняет команду **молча в телеметрии, но с текстом в USB**. Правила:

- `w` требует заданного нуля (`cal=1`), иначе `w: ноль не задан — сначала k …`;
- цель `w` вне ±100° отклоняется;
- поворот запрещён, пока включена тяга (`руль заблокирован: включена тяга`);

- смена режима во время движения/калибровки требует `s`;
- `v` требует свежих команд и работает без нуля; `t` требует ноль руля;
- любая команда тяги старше 650 мс → выход 0 и тормоз;
- `ph 1` требует `pm 1`, ноль и выполненную `q`; при ошибке памяти полного нуля (`zfault=1`) отклоняется.

---

2. Направление контроллер → ПК

Три типа строк. Первый символ определяет тип: `#` — человеку, `@` — машине.

2.1. Сообщения для человека: `# XX: текст`

```
# FL: --- КОЛЕСО FL: руль v26, USB 115200; ноль не задан — k или z ---
# FL: Калибровка ЗАВЕРШЕНА! Флажок: 12.3°, ноль: 0.0°
# FR: Потенциометр: расхождение 31.08° — доводка отключена; проверьте колесо или q
```

Формат: `# <РОЛЬ>: <UTF-8 текст>\r\n`. Сервер складывает эти строки в журнал экрана/пульта. **Разбирать их программно нельзя** — для логики есть @F/@TLM.

2.2. Короткая телеметрия: `@F`

Кадр формируется обычно каждые **30 мс**; изменение состояния (тормоз, тяга, руление, свежие фронты SC) запрашивает кадр раньше, но не чаще одного раза в **20 мс**. Длина строки ≤ 160 байт.

```
@F <fw> <role> <deg_c> <tgt_c> <status> <drive> <dmax> <sv> <rpm_x10> <hz> <ppr> <count> <recent> <brake_id>
    <neutral_id> <neutral_lock> <live_id> <live_seq> <ms> <pa> <crc>
```

Пример — настоящая строка симулятора прошивки, сумма верная:

```
@F 26 2 81 2500 12363 20 100 36 0 0 90 1 1 0 0 0 42 3 21 32767 2BFF
```

В строке fw26 **22 слова**: @F, двадцать чисел и CRC. До fw26 поля ра не было, и слов было 21; сервер, не знающий fw26, такой кадр отбрасывает — поэтому сначала обновляется сервер, затем платы.

№	Поле	Тип	Значение
1	`fw`	u8	версия протокола прошивки, сейчас 26
2	`role`	u8	1=FL, 2=FR, 3=RL, 4=RR, 0=роль в скетче не задана
3	`deg_c`	i32	угол колеса **по счётчику шагов**, сотые доли градуса (1250 = 12.50°)
4	`tgt_c`	i32	целевой угол, сотые доли градуса
5	`status`	u16	биты состояния, см. §3
6	`drive`	i16	заданная тяга, −100…+100 % (шкала Servo)
7	`dmax`	u8	предел процентной тяги (`t`), сейчас 100
8	`sv`	u8	значение, переданное в `Servo.write` (0…180)
9	`rpm_x10`	u32	обороты по SC в **десятых** (600 = 60.0 об/мин)
10	`hz`	u16	принятая частота фронтов SC, Гц
11	`ppr`	u16	принятых переходов на оборот (TACH_EDGES_PER_REV)
12	`count`	u32	непрерывный счётчик фронтов SC по модулю 2^{32}
13	`recent`	u8	1 — фронты были недавно (окно 300…1500 мс)
14	`brake_id`	u32	подтверждённый id тормоза `b N`, 0 — нет
15	`neutral_id`	u32	устаревшее поле нейтрали (fw18…24), с fw25 всегда 0
16	`neutral_lock`	u8	устаревшая блокировка нейтрали, с fw25 всегда 0

№	Поле	Тип	Значение
17	live_id	u32	id живой сессии руления на ходу (j), 0 — нет
18	live_seq	u32	последний применённый номер пакета j
19	ms	u32	millis() платы по модулю 2^{32}
20	pa	i32	**fw26:** угол по потенциометру A0, сотые доли градуса; 32767 — пригодного показания нет (§2.5)
21	crc	hex16	CRC16/Modbus предыдущих байт строки

2.3. Полная телеметрия: @TLM

Каждые **200 мс** (совместимость со старыми серверами). Длина ≤ 384 байт. Поля — ключ=значение в фиксированном порядке:

```
@TLM mod=FR deg=0.81 tgt=25.00 moving=1 cal=1 cycle=0 calib=0 opto=1 home=manual
     err=0 drive=20 fw=26 pot=0 pm=1 ph=0 zfault=0 tach=0 dmax=100 sv=36 duty=200
     sc_rpm=0.0 sc_hz=0 sc_ppr=90 sc_count=1 sc_recent=1 brake_id=0 neutral_id=0
     neutral_lock=0 live_id=42 live_seq=3 t=21 pa=32767 crc=A1EA
```

(На плате это одна строка; здесь она перенесена для чтения.)

Поле	Значение
mod	роль: FL / FR / RL / RR
deg, tgt	угол по счётчику шагов и целевой угол, ° (-135.00…135.00)
moving	1 — руль едет
cal	1 — ноль задан
home	откуда ноль: opto / manual / none
calib	1 — идёт калибровка
opto	1 — датчик D18 сейчас закрыт (колесо над флажком)
cycle	1 — идёт качание с
err	код ошибки калибровки: 0 — нет; 1…6 см. firmware/README.md §5
drive	заданная тяга, %
fw	версия прошивки/протокола
pot	1 — режим датчика включён, есть калибровка q и ноль, показание A0 не на краю шкалы: доводка работает
pm	режим потенциометра: 1/0, -1 — прошивка не поддерживает переключение
ph	активное удержание угла: 1/0
zfault	1 — сохранённая поправка полного нуля повреждена
tach	всегда 0 (старый канал одометрии выключен)
dmax	предел старой процентной тяги
sv	Servo.write (0…180)
duty	то же в промилле шкалы Servo (1000 = write(180))
sc_rpm	обороты по SC, об/мин
sc_hz	частота фронтов SC, Гц
sc_ppr	переходов на оборот
sc_count	счётчик фронтов по модулю 2^{32}

Поле	Значение
`sc_recent`	1 — фронты были недавно
`brake_id`, `neutral_id`, `neutral_lock`, `live_id`, `live_seq`	подтверждения сессий (см. §2.2)
`t`	`millis()` платы
`pa`	**fw26:** угол по потенциометру A0, сотые доли градуса; 32767 — пригодного показания нет (§2.5)
`crc`	CRC16/Modbus строки до `" crc="`

2.4. Ответ на запись полного нуля: @Z

```
@Z <fw> <role> <request_id> <status> <saved_steps> <proposed_steps> <crc>
```

Поле	Значение
`fw`	версия прошивки (23+)
`role`	1…4
`request_id`	id из `zq <id>` / `zc <id>`
`status`	0 — успех/подтверждение, иначе код ошибки
`saved_steps`	сохранённая поправка в шагах
`proposed_steps`	предложенная поправка в шагах (для `zc`)
`crc`	CRC16/Modbus

Кадр повторяется недолго; после перезагрузки платы id запросов забываются.

2.5. Угол по потенциометру: поле pa (fw26)

До fw26 плата передавала один угол — deg, расчёт по счётчику шагов. После калибровки `q` счётчик привязывается к потенциометру, но сервер не мог отличить «угол измерен датчиком» от «угол посчитан по шагам». В fw26 измерение вынесено в отдельное поле того же кадра, под ту же контрольную сумму:

Значение `pa`	Смысл
`-18000…18000`	угол колеса по потенциометру A0, сотые доли градуса
`32767`	**не угол**: пригодного показания сейчас нет

32767 передаётся, когда не выполнено хотя бы одно условие: режим датчика включён (`pm 1`), есть калибровка `q` и ноль руля, показание A0 не на краю шкалы АЦП, не идёт калибровка, нет ошибки памяти полного нуля, расчётный угол не вышел за ±180°.

Правила для сервера:

- deg остаётся расчётом по шагам и по-прежнему используется в проверках безопасности — источник обратной связи молча не меняется;
- pa показывается оператору как измеренный угол, а при 32767 экран обязан явно написать, что угол — по шагам;
- платы fw24–25 поле не передают: для них угол всегда по шагам, даже после `q`;
- вход A0, «плавающий» в середине шкалы при обрыве, выглядит правдоподобно — ни pa, ни CRC не доказывают исправность проводки и положение колеса.

Вместе с pa fw26 исправляет две вещи, видимые по протоколу. Наклон калибровки `q` теперь восстанавливается из EEPROM с любым знаком (раньше отрицательный терялся, и после перезапуска `pot=0`). А при уходе A0 на край шкалы или ошибке памяти нуля плата сама снимает бит `POT_HOLD` и прерывает начатую поправку.

3. Биты состояния

Бит	Маска	Смысл
0	0x0001	MOVING — руль едет
1	0x0002	ZERO — ноль задан (cal)
2	0x0004	CALIB — идёт калибровка
3	0x0008	OPTO — датчик D18 закрыт
4	0x0010	CYCLE — качание
5	0x0020	HOME_OPTO — ноль получен по оптодатчику
6	0x0040	V2 — признак кадра этой версии
7	0x0080	POT — режим датчика, калибровка q и ноль есть, A0 не на краю шкалы: доводка работает
8–11	0x0F00	код ошибки калибровки err (сдвиг 8)
12	0x1000	POT_MODE — режим «с потенциометром»
13	0x2000	POT_MODE_SUPPORTED — прошивка умеет pm
14	0x4000	FULL_ZERO_FAULT — сохранённая поправка не заслуживает доверия
15	0x8000	POT_HOLD — активное удержание угла включено

---

4. CRC16/Modbus

Полином 0xA001, инициализация 0xFFFF, байты — как есть, без инверсии. Считается по **всем байтам строки до суффикса**, то есть без `" crc="` (в @TLM и во всех кадрах модуля датчиков) и без ведущего пробела перед четырьмя hex-цифрами (в @F, @Z).

```
def crc16(data: bytes) -> int:
    crc = 0xFFFF
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc
```

Сервер обязан **отбрасывать кадр с неверной CRC** и не обновлять по нему время свежести: иначе битая строка с cal=1 откроет движение. CRC защищает только от случайных ошибок передачи; это **не аутентификация и не E-STOP**.

---

5. Таймауты и частота

Событие	Значение	Реакция
Нет команды тяги	**650 мс**	drive = 0, BRAKE активен, STOP снят
Потеря @F	по настройке экрана (обычно 1 с)	модуль считается офлайн, команды не шлются
@F при изменении состояния	не чаще **20 мс**	срочный кадр
@F в покое	**30 мс**	обычный кадр
@TLM	**200 мс**	совместимость
Таймаут калибровки	120 с	err=5, ноля нет

Скорость линии **115200 бод 8N1** оставлена намеренно: 250000 бод возможны, но требуют согласованной правки обеих сторон и проверки конкретного USB-UART.

---

6. Со стороны ПО (Zapret)

Весь бортовой софт — в соседнем репозитории danilka-revin/Zapret:

- `slam_gui/backend.py` — `MasterSource`: открывает 4 порта колёс, парсит `@F`/`@TLM`, адресует команды по имени колеса, следит за свежестью и подтверждениями;
- `robot_control/rus_core/protocol.py` — сборка/разбор строк протокола и CRC16, общий для сервера и тестов;
- `robot_control/sensor_module.py` — разбор кадров модуля датчиков (§7) и клиент его USB-порта;
- `tests/test_firmware.py` и `tests/test_sensor_module.py` в Zapret читают `FIRMWARE_PROTOCOL` и `SENSOR_PROTOCOL` **из этих скетчей** и сверяют с сервером; симуляция прошивки (`firmware/firmware_sim` здесь) отдаёт настоящие кадры через PTY.

Порядок изменений: сначала этот документ и скетч, потом парсер и тесты в Zapret. Порядок обновления на роботе обратный: **сначала сервер, затем платы** — новый сервер понимает старые кадры, а старый сервер новые кадры `@F` отбрасывает.

---

7. Модуль датчиков

Отдельная плата Arduino Mega 2560 на своём USB, скетч `firmware/sensor_module/sensor_module.ino`. Физика та же, что у колёс: **115200 8N1**, строки ASCII, перевод строки в конце, CRC16/Modbus из §4. Плата управляет светом, замком грузового отсека и картами доступа и отдаёт курс; к приводу она отношения не имеет и остановить робота не может. Распиновка и правила включения — `firmware/README.md` §16 и `ELECTRICAL_AND_PINOUTS.md` §6.

Параметр	Значение
Версия протокола	**4** (`SENSOR_PROTOCOL`, поле `fw` кадра `@S`)
Телеметрия `@S`	каждые 200 мс (5 Гц) и сразу после команды, меняющей состояние
Подтверждение	`@ACK` на каждую принятую строку
Сторож связи	нет команд **10 с** → `link=0`, замок, открытый на время, закрывается
Сторожевой таймер платы	2 с
Порт на сервере	`RC_SENSOR_PORT`, задаётся вручную

7.1. Команды ПК → плата

Команда	Назначение
`ping` / `stat`	прислать @S и подтверждение
`?` / `help`	справка строками `# SENSORS: …`
`light head|stop|left|right|dim on|off`	один канал света: фары, стопы, левый и правый поворотник, габариты
`light all on|off`	все каналы сразу
`dim <0…255>`	яркость габаритов на ленте
`strip auto|off|floor|wave`	режим адресной ленты
`lock open [1…10]`	открыть замок на время, с; без числа — 3 с
`lock hold`	открыть и держать до `lock close` или карты
`lock close`	закрыть замок
`card list`	переслать список карт кадрами @CARDINFO
`card learn [admin|user]`	записать следующую приложенную карту
`card add <UID> [admin|user]`	записать карту по UID (8 hex-знаков)
`card role <UID> admin|user`	сменить роль карты
`card del <UID>` / `card clear`	удалить карту / все карты
`cargo load|unload [1…240]`	открыть окно погрузки или выгрузки, мин; по умолчанию 15
`cargo end`	закрыть окно
`mag cal start|stop|reset`	калибровка компаса (оборот робота на 360° между `start` и `stop`)

Неизвестная строка получает `@ACK … ok=0`; состояние платы не меняется.

7.2. Кадры плата → ПК

```
@S fw=4 hdg=12345 pitch=-120 roll=35 gz=12 press=69964 temp=150 alt=301674 lock=0 light=5 dim=80 strip=1
    nleds=73 cargo=0 rfid=0 cards=2 link=1 err=0 t=123456 crc=ABCD
@CARD uid=A1B2C3D4 ok=1 act=open role=admin why=none cards=2 t=1234 crc=ABCD
@CARDINFO idx=1 uid=A1B2C3D4 role=admin total=2 t=1234 crc=ABCD
@CARDINFO end=1 total=2 t=1234 crc=ABCD
@EVT kind=lock open=1 t=1234 crc=ABCD
@EVT kind=cargo state=load t=1234 crc=ABCD
@ACK id=3 cmd=lock ok=1 t=1234 crc=ABCD
# SENSORS: карта A1B2C3D4 записана (администратор)
```

Суммы ABCD в примерах условные. Строки `# SENSORS: …` — для человека, разбирать их программно нельзя.

@S **— телеметрия.** Поля идут в фиксированном порядке:

Поле	Значение
`fw`	версия протокола платы, сейчас 4
`hdg`	курс, сотые доли градуса, 0…35999; наклон скомпенсирован акселерометром
`pitch`, `roll`	тангаж и крен, сотые доли градуса
`gz`	угловая скорость вокруг вертикальной оси, десятые доли °/с
`press`	давление, Па (0 — замера ещё нет)
`temp`	температура, десятые доли °C
`alt`	высота над уровнем 101325 Па, см
`lock`	1 — замок отсека открыт

Поле	Значение
`light`	биты света: 1 фары, 2 стопы, 4 левый, 8 правый поворотник, 16 габариты; у поворотников — с учётом фазы мигания
`dim`	яркость габаритов, 0…255
`strip`	1 — адресная лента смонтирована, 0 — прошивка её не трогает
`nleds`	число диодов ленты (на роботе 73)
`cargo`	окно: 0 закрыто, 1 погрузка, 2 выгрузка
`rfid`	последнее событие карты: 0 нет, 1 пустили, 2 отказали, 3 записали, 4 удалили
`cards`	число карт в памяти (до 12)
`link`	0 — команд от сервера нет дольше 10 с
`err`	биты отказов: 1 MPU6050, 2 HMC5883L, 4 BMP180, 8 RC522, 16 EEPROM, 32 замок, 64 шина I²C
`t`	`millis()` платы
`crc`	CRC16/Modbus строки до `" crc="`

`@CARD` **— приложили карту.** `uid` — 8 hex-знаков; `ok` — 1 пустили, 0 нет; `act` — что сделала плата: `open`, `close`, `enroll` (карта записана) или `deny`; `role` — `admin`, `user` или `none`; `why` — причина отказа: `none`, `unknown` (карта не записана), `cargo` (карта пользователя вне окна погрузки), `full` (список заполнен); `cards` — карт в памяти.

`@CARDINFO` **— список карт.** По кадру на карту (`idx`, `uid`, `role`, `total`) и завершающий кадр `end=1 total=N`. Приходит при старте платы, по команде `card list` и после изменения списка командой: `card add`, `card role`, `card del`, `card clear`. Когда карту записывают прикладыванием после `card learn`, список не пересылается: о новой карте сообщает кадр `@CARD … act=enroll` с новым числом `cards`.

`@EVT` **— события.** `kind=lock open=0|1` — замок закрылся или открылся; `kind=cargo state=load|unload|end` — окно погрузки или выгрузки открыто либо закрыто.

`@ACK` **— подтверждение.** `id` — порядковый номер принятой платой строки (с момента сброса), `cmd` — первое слово команды, `ok` — 1 выполнена, 0 отклонена.

7.3. Чего в протоколе 4 нет

Концевик грузового отсека не подключён, поэтому плата **не сообщает, открыта ли дверь**: соответствующего поля в `@S` и события двери нет, вывод D8 свободен. Запрета движения при открытом отсеке тоже нет. Сервер Zapret понимает и платы с протоколом 3, в кадрах которых поле двери было: оно игнорируется.

Доступ по картам — удобство, а не система контроля доступа: UID карты Mifare Classic копируется, CRC защищает только от помех на линии.

---

8. Другие устройства на USB бортового компьютера

Эти устройства описанным здесь протоколом **не пользуются**; они перечислены, чтобы было видно, какие порты заняты и чем. Работу с ними ведёт софт Zapret.

Устройство	Подключение	Обмен	Настройка в Zapret
Лидар LDS-01 (HLS-LFCD2)	USB-UART адаптер лидара	230400 бод, собственные двоичные кадры лидара	`RC_LIDAR_PORT`, `RC_LIDAR_PROTOCOL=lds01`
BMS аккумулятора (JBD)	USB-UART на диагностический разъём: TX→RX, RX→TX, GND→GND	9600 8N1, запрос-ответ JBD (регистры `0x03`, `0x04`), только чтение	`RC_BMS_PORT`

Устройство	Подключение	Обмен	Настройка в Zapret
Чековый принтер	отдельный USB-UART	9600 8N1 по умолчанию, текст или ESC/POS	`RC_PRINTER_PORT`
4 × камера Logitech C930e	USB (UVC)	видеопоток V4L2	`RC_CAMERA_FRONT`, `RC_CAMERA_FRONT_RIGHT`, `RC_CAMERA_REAR`, `RC_CAMERA_REAR_RIGHT`
Kinect 360	USB и отдельный блок питания	глубина через libfreenect / OpenNI2	`RC_KINECT_PORT`

Все последовательные порты назначаются **вручную**; автопоиска нет. Имена `/dev/ttyUSB*` и `/dev/ttyACM*` после переподключения могут поменяться, поэтому в настройках указывают пути `/dev/serial/by-id/…`. Один порт не открывают два процесса, а любое открытие порта платы Arduino может её перезагрузить.